

Business Continuity & Disaster Recovery Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC7.5 (Recover from disruptions) · CC9.1 (Risk mitigation for business disruptions) · HIPAA §164.308(a)(7) Contingency plan · ISO 27001 A.5.30 (ICT readiness for business continuity) **Companion documents:** Incident Response Policy §7 · Encryption Policy §7 · Data Retention & Deletion Policy §7.

1. Purpose

State what has to keep working when things break, how long we're willing to be down, and the specific playbook to get back. Every RTO/RPO here is one this team can actually meet with the operational reality we have — not aspiration, not marketing.

2. Business-critical services

Ranked by what actually stops the business if it's down.

#	Service	Stops what if down	Owner
1	Local station on customer machines	Every customer's product	Customer (we can't affect this once shipped; recovery = reinstall)
2	<code>railcall.ai/install.sh</code> + <code>railcall_station.tar.gz</code> mirror	New signups + reinstalls	VPS <code>157.230.177.45</code> + <code>git pull</code> on the box
3	Gateway <code>railcall-core.onrender.com</code>	Paid tier: mint/activate/checkin/webhook	Render
4	Gateway Postgres	All paid-tier state — subscriptions, seat reservations, meter	Render managed
5	Stripe	New checkouts, subscription lifecycle events	Stripe
6	<code>railcall.ai</code> marketing site	Sign-up conversion, docs pages	Same VPS as #2
7	GitHub Releases + repo	Cutting new station releases, <code>install.sh</code> distribution	GitHub
8	Anthropic API (Probo only)	Our own compliance drafting velocity	Anthropic (dev-only, no customer impact)

Deliberately absent: any "internal SaaS" (there is none), any "employee laptop" (Sami's box has the issuer seed backup but is not itself business-critical because the seed is also in 1Password + paper).

3. Recovery objectives

RTO = how long a service can be down before it's a problem. RPO = how much data loss is acceptable at recovery.

Service	RTO	RPO	Justification
Local station (per customer)	15 min	0 (data lives on customer's box, not affected by our outages)	Customer runs the installer; nothing on our side to restore
<code>install.sh</code> + tarball mirror	4 h	0 (installer/tarball are static, versioned; recover from GitHub Release + git pull)	Blocks new signups; existing customers unaffected
Gateway (paid- tier code path)	4 h	0 (stateless code; state is Postgres)	Paid customer already activated is unaffected; new mints blocked
Gateway Postgres	24 h	24 h (Render managed backup cadence)	Losing 24 h of <code>consumers</code> rows = losing recent signups (recoverable from Stripe webhook replay); losing <code>seat_reservations</code> is self-healing on next checkin
Stripe (their outage)	Their SLA	Their SLA	Nothing we can restore; documented for transparency
Marketing site	24 h	0 (git-versioned)	Not paid-tier blocking; conversion loss only
GitHub	Their SLA	Their SLA	Our critical dependency; a full GitHub outage delays station releases until they recover or we mirror elsewhere (§5.6)
Anthropic (Probo)	N/A	N/A	Dev tooling; no customer impact

4. Backups + snapshots — what we can restore from

Data	Where the backup lives	Backup cadence	Restore verified?
Gateway Postgres	Render managed backup surface	Render default	Not verified end-to-end by us; relies on Render restore procedure. Action item: run one real restore drill against a staging DB before EOY 2026.
VPS <code>157.230.177.45</code>	DigitalOcean snapshots	Weekly, 4-week retention	Not verified. Action item: annual snapshot-restore drill.
Issuer seed	1Password + offline paper in Sami's safe	Manual, on rotation	Verified — the mint tool's <code>check</code> subcommand recomputes the pubkey from the stored seed so a restored seed can be validated without deploying.
Source code	GitHub (patl4588.org)	Every push	Verified continuously — every fresh clone IS a restore.
Signed release tarballs	GitHub Releases + railcall.ai mirror	On every station cut	Verified — the release ceremony byte-compares the downloaded tarball against the local build before pinning <code>install.sh</code> .
This compliance folder	Git (railcall-core <code>compliance/</code>) + Probo	Every commit / every policy edit	Verified — Git IS the retention layer.

5. Disaster runbooks

Each runbook is scoped by "what failed", not by "what caused it." The cause matters for the post-mortem, not for the restore.

5.1 Issuer signing seed lost (no compromise suspected)

Cross-reference: Encryption Policy §7 + Incident Response Policy §7.1.

RTO: 8 hours (station release cycle is the bottleneck). **RPO:** 0 for existing entitlements (they keep verifying against the pinned key until expiry). New mints blocked until the new key is in the pin.

Steps: 1. Mint new keypair: `python3 tools/mint_issuer_keypair.py mint --seed-out ~/railcall-issuer-seed-YYYYMMDD.txt`. Back up to 1Password + paper BEFORE proceeding. 2.

Edit `ISSUER_PUBKEY_HEX` in `railcall-engine/workbench/primitives/entitlement.py`. Commit + push. 3. Cut new station release (`station-vN+1`): overlay engine into `~/.railcall/station/`, run `bash scripts/build_station_tar.sh`, verify leak gate + fresh-install smoke. 4. `gh release create station-vN+1` with the tarball. Byte-compare downloaded asset against the local build. 5. Update `STATION_SHA` + `STATION_URL` in `railcall-core/install.sh` and `install.ps1`. Push. 6. Mirror to `railcall-contrib/website-v2/public/` (both installer + tarball). Push. 7. `ssh sami@157.230.177.45 && cd ~/railcall-contrib && git pull && pm2 restart railcall-website`. 8. Verify `curl https://railcall.ai/install.sh` returns new sha, and public tarball sha matches. 9. Set `RAILCALL_ISSUER_SEED` on Render service (Encryption Policy §4.2 — direct service-level, not env group). 10. Verify `curl https://railcall-core.onrender.com/v1/issuer/pubkey` returns the new pubkey. 11. Email every paying customer: "reinstall via `curl -fsSL https://railcall.ai/install.sh | bash`, then `railcall activate` to re-mint your entitlement."

5.2 Render gateway complete outage

RTO: 4 hours (dependent on Render's own recovery). **RPO:** 0 (state is durable in managed Postgres).

Steps: 1. Confirm scope via <https://status.render.com>. 2. If our service specifically is down (not Render broadly): check Render dashboard for failed deploys, roll back to the last known-good deploy. 3. If Render broadly is down: post to Discord + Twitter. Nothing else we can do — this is Render's runbook. 4. Free-tier installs are unaffected (they don't depend on the gateway). Communicate this in the outage notice so free users don't panic. 5. Paid customers with active entitlements are unaffected until expiry. If an entitlement expires during the outage, extend it manually via email after service returns. 6. Post-outage: replay any Stripe webhooks that failed during the window (Stripe retries automatically for 72 h).

5.3 Gateway Postgres data loss

RTO: 24 hours. **RPO:** 24 hours (Render managed backup cadence).

Steps: 1. Immediately halt new writes (put gateway in maintenance mode via a feature flag or manual deploy of a maintenance-page image). 2. Restore latest Render backup to a NEW DB instance. 3. Point gateway env `DATABASE_URL` at the restored instance. Redeploy. 4. Replay Stripe webhooks for the period between backup point and outage. `checkout.session.completed` events are idempotent on `cs:<session_id>` so replay is safe. 5. Seat reservations are self-healing — every install pings every 6 h, table reconstructs within a TTL cycle. 6. Send affected customers (identified by absence of expected `consumers` row) a mint-retry link so they get their entitlement re-issued.

5.4 VPS `157.230.177.45` unreachable or lost

RTO: 4 hours (DNS TTL is the bottleneck). **RPO:** 0 for installer/tarball (they live in Git + GitHub Releases).

Steps: 1. Confirm scope via DO console. If VPS unrecoverable: provision a new droplet from the latest DO snapshot. 2. If snapshot restore is fine: point `railcall.ai` A record at the new IP. DNS

propagates in ≤ 5 min if TTL is low. 3. If snapshot unavailable: fresh droplet, install Node + pm2, `git clone git@github.com:patl4588/railcall-contrib.git`, checkout `feat/website-v2`, `npm ci` && `npm run build`, `pm2 start ...`. ~1 hour. 4. Add Sami's SSH key + Pat's root key. Verify installer + tarball are byte-identical to the GitHub Release copy before flipping DNS. 5. Post-recovery: send a "we moved boxes" note only if anyone noticed.

5.5 Stripe outage

RTO: Stripe's SLA. **RPO:** Stripe's SLA.

Nothing we can do besides communicate. Existing subscribers keep being charged by Stripe independently. New checkouts blocked until they're back. Add a banner to the pricing page pointing to Discord for updates.

5.6 GitHub outage (or GitHub compromise affecting our org)

RTO: their SLA for outages; 8 h to switch to mirror on compromise.

Steps for outage: 1. Wait for GitHub to recover. Nothing we can do; every dev depends on GitHub for something.

Steps for org compromise (unauthorized push to patl4588/railcall-core or *-engine): 1. Rotate every push token. Enforce SSH-key-only + MFA everywhere. 2. Force-push last known-good commit from any local checkout that predates the compromise. 3. Cut a new station release ONLY after auditing every commit since the compromise window. 4. Notify anyone who installed from a compromised commit window — if the tarball on the release DIFFERS from what our local build produces, treat this as a supply-chain event per Incident Response Policy §7.7.

5.7 Sami unavailable (primary operator down)

Sober edge case for a two-person team. Steps to make it survivable:

1. Every root secret Sami holds must also be in a place Pat can reach IF the situation warrants — 1Password shared vault, or sealed envelope in a physical location Pat knows about. Currently: 1Password backup of issuer seed exists; sealed physical backup is an **action item**.
2. Pat has SSH access to the VPS with a root key. Verify annually that his key still works.
3. GitHub org owner is Pat; Sami is admin. If Sami is unavailable, Pat can still merge/tag/release.
4. Render is admin-shared. Stripe is admin-shared (as of 2026-07-21).
5. Anthropic account is Sami's — a Pat-usable API key is an **action item**.

Documented explicitly: if Sami is unavailable for > 24 h and no incident is in progress, business continues without material impact. If Sami is unavailable DURING an active SEV-1, Pat + a Signal call to Claude Code (which has the runbooks) is the fallback.

5.8 Sami's laptop lost / stolen

RTO: 4 hours (mostly re-provisioning). **RPO:** 0 (nothing customer-critical lives ONLY on Sami's laptop).

Steps: 1. From another box, revoke Sami's SSH keys everywhere (GitHub, VPS, Render). 2. Rotate 1Password master passphrase (whole vault re-encrypted). 3. Rotate every API key Sami had — Stripe (all of them), Anthropic, GitHub PATs, Render CLI token if any. 4. Verify the issuer seed is still safe (1Password is E2E encrypted so a laptop grab doesn't reveal it; paper backup in the safe is unaffected). 5. Provision a new box. Sami reinstalls the dev tooling from public sources — no proprietary tooling.

6. Testing

- **Tabletop exercises:** one per quarter, alternating scenarios from §5. Log as SEV-4 (tabletop) incidents per Incident Response Policy §10.
- **Live drill:** annual issuer-seed rotation (§5.1) — the natural exercise because it's the highest-blast-radius runbook AND it's periodically valuable to actually do (freshness of the ceremony).
- **Backup restore drill:** annual Postgres restore into a staging DB (§4 action item).

7. Business impact analysis (BIA) — summary

Financial exposure of each scenario, roughly:

Scenario	Financial impact per day of outage	Reputational impact
§5.1 issuer seed lost	Zero direct revenue impact (existing subs continue); reputation risk if we handle the notification badly	Medium — every paying customer has to act
§5.2 gateway outage	~\$daily-signups-cost — depends on signup rate	Low — free tier unaffected
§5.3 Postgres data loss	~1 day of signups lost + customer trust hit	Medium — customers noticing missing account state
§5.4 VPS lost	~\$signups-during-outage + install-time friction	Low — mostly transparent to existing users
§5.5 Stripe outage	Zero direct (they hold their own state); new signups blocked	Low — attributable to Stripe
§5.6 GitHub compromise	Existential if pinned code is tampered undetected	Very high
§5.7 Sami unavailable	Slowed velocity; no direct outage	Depends on duration
§5.8 laptop lost	Zero if credentials rotate cleanly; existential if issuer seed also compromised in transit	Medium

8. Review

Whole policy reviewed at each station release + at least annually. Action items in §4 (backup restore drill) and §5.7 (Pat operational continuity) are tracked in Probo and re-verified quarterly.

9. Related documents

- Incident Response Policy §7 — how we detect + triage what §5 recovers from.
- Encryption Policy §7 — the key-lifecycle piece of §5.1.
- Data Retention & Deletion Policy §7 — what backups exist and their retention.
- Access Control Policy §6 — who has the credentials each recovery step requires.
- Vendor Management Policy §3 — the per-vendor posture that determines each RTO/RPO.